

Certified Workload-Window Plan Selection for Hybrid Vector Search under SLOs

Yanxiao Li¹ , Junhao Wei¹ , Yifu Zhao¹ , Haochen Li¹ , Xu Yang¹ , Sio-Kei Im² , Yapeng Wang^{1,*} 

¹Faculty of Applied Sciences, Macao Polytechnic University, Macao SAR, China

²Macao Polytechnic University, Macao SAR, China

*Corresponding author: Yapeng Wang (e-mail: yapengwang@mpu.edu.mo)

Abstract—Hybrid vector search engines expose many physical plans—lexical retrieval, dense retrieval, learned-sparse retrieval, late interaction, fusion, metadata filtering, and reranking. Selecting a plan under a latency SLO is a query-optimization problem, yet static policies and per-query routers give little assurance that the chosen plan stays feasible once the next workload window shifts. We propose CWC, a certified workload-window compiler for hybrid-search plan selection. Given a labeled historical workload, an unlabeled incoming workload window, and telemetry from candidate operators, CWC partitions queries into workload cells, binds each cell to a physical plan, and abstains to a feasibility-best fallback when the selected plan is fragile. Its certificate is a finite-sample, distribution-free upper bound on the evaluation-window SLO-violation rate under a specified within-cell shift budget; in experiments, a conservative plug-in budget upper-bounds the realized violation in every run. On a 45-block study over SciFact, FiQA, and NFCorpus with BM25, E5, Qwen3-Embedding, SPLADE, ColBERTv2, fusion, and reranking actions, CWC ranks first on constrained utility, significantly outperforms four strong static operators, and statistically ties calibration-aware selectors while adding a deployment certificate they lack. At the binding budget it cuts realized SLO violations from about 11% to about 0.1% at roughly 4% nDCG cost, and a scaling study shows the certified bound tightens with per-cell calibration size, as the theory predicts.

Index Terms—Hybrid search, vector search, query processing and optimization, physical plan selection, workload-aware optimization, service-level objectives, distribution-free risk control

I. INTRODUCTION

Modern data systems increasingly embed hybrid and vector search. Production hybrid-search stacks combine BM25 [1], dense vector search [2], learned-sparse retrieval [3], late interaction [4], reciprocal-rank fusion [5], metadata-filter placement, and neural reranking [6]. This is the operator menu that the retrieval literature and current vector engines expose in general; we do not model one particular deployment, and the action catalog we actually execute is listed in Table II. These are *physical plans* with different latency profiles, filter behavior, and failure modes, so the systems question is a query-optimization one: which physical plan should serve a workload under a latency SLO and a filter constraint, and, critically, can that choice be *trusted* once the workload drifts

from what was seen at calibration. A plan that is fast on yesterday’s queries can silently start violating the latency SLO when today’s workload shifts toward harder, higher-selectivity queries. In deployment the quantity that matters is the *rate* at which the chosen plan breaches the SLO over a serving window: this rate drives SLA penalties and admission decisions, yet current selectors set it by hoping a plan generalizes, with no guarantee before execution. Bounding it ahead of time—so an operator can audit, and admit or hold back, a plan before it serves traffic—is the gap we close.

Most current strategies choose at the wrong unit and offer no such trust. Fixed recipes such as BM25-only, dense-only, fixed fusion, or fixed reranking depth are simple but brittle: the best plan changes with selectivity, budget, and query mix. Per-query routers are more adaptive, but they decide from isolated query features, and the documented unreliability of query performance prediction makes per-query selection fragile in exactly the constrained regime we care about. Neither family, however, says anything about *how often the deployed choice will violate the SLO on the next, possibly shifted, workload window*. An arriving workload window, however, can be observed before execution: it reveals stable regions where the same plan is repeatedly good, and it lets a system bound its own risk instead of merely hoping the chosen plan generalizes.

This paper treats hybrid-search plan selection as *workload-window physical-plan compilation with an auditable deployment guarantee*. The input is a labeled training workload, an unlabeled evaluation workload window, and telemetry from candidate operators; the output is a compiled design—queries partitioned into plan cells, a physical plan bound to each cell, a cell-level plan frontier—together with a *distribution-free, finite-sample certificate* on the eval-window SLO-violation rate that holds under a bounded amount of workload shift. The certificate is operational: when a cell’s plan is not certifiably safe, the compiler abstains to a certifiably feasible fallback. The operators themselves are unchanged; what is new is that the optimizer certifies *which plan to deploy, and when to abstain*, the way a database physical-design tool reasons about a workload rather than a single query. To our knowledge no prior hybrid-search method provides such a guarantee, and none of our baselines provides one as used.

Our system is CWC, the *Certified Workload-window Compiler*. CWC first embeds train and evaluation queries using

observable workload telemetry—cheap retrieval probes, score-shape features, branch overlap, filter survival, and latency signals—then partitions the workload into plan cells and binds each cell to the action with the best training constrained utility, so each evaluation query inherits the plan of its cell. On top of this it adds an abstain rule and a deployment certificate, both developed later in the paper. We write WCR for the *un-guarded* variant (the same compiler without the abstain rule); it is used only as an ablation and in the preliminary single-collection pilot and cross-dataset audit, and when no cell abstains $CWC \equiv WCR$. The method is transductive in the systems sense: it uses unlabeled telemetry from the current workload window, but never evaluation relevance labels.

Our canonical evidence (Section VII-A) is a 45-block study over SciFact, FiQA, and NFCorpus with real external operators under a controlled bounded workload shift; a single-collection NFCorpus pilot and a 360-block cross-dataset audit corroborate it as robustness, not the headline.

The claim is intentionally scoped: optimizer-level gains plus an auditable deployment guarantee under a fixed action catalog, not universal superiority over every retriever stack. In particular CWC *ties* the strongest calibration-aware selectors on mean utility; its differentiator there is the certificate, not raw rank.

This paper makes three contributions.

- 1) **Abstraction.** We formulate hybrid-search plan selection as *certified workload-window physical-plan compilation*: an unlabeled workload window is partitioned into plan cells, each cell is bound to an executable physical plan, and each deployment decision carries an auditable risk guarantee.
- 2) **Guarantee.** We prove a distribution-free, finite-sample certificate (Proposition 2) that upper-bounds the eval-window SLO-violation rate under a bounded within-cell shift budget, made operational through CWC, a guarded compiler that abstains to a feasibility-best fallback on fragile cells; a density-ratio upper-confidence-bound variant removes the assumed-shift caveat. To our knowledge this is the first such guarantee at the *workload-window* granularity for hybrid-search plan selection.
- 3) **Evaluation.** On a 45-block SciFact+FiQA+NFCorpus workload with BM25, E5, Qwen3-Embedding, SPLADE, ColBERTv2, fusion, and reranking actions, CWC is average-rank first on constrained utility, Holm-beats four strong static operators, and ties calibration-aware selectors while adding a certificate they lack, cutting realized SLO violations from $\approx 11\%$ to $\approx 0.1\%$ at $\approx 4\%$ nDCG cost; under an SLA admission-control objective it separates from those selectors (rank 1.44, Holm-beating all six baselines), and a scaling study confirms the bound tightens with per-cell calibration size. Ablations and a 360-block audit isolate the source of the gains.

II. RELATED WORK

One framing prevents most confusion with prior art: we do not propose another retriever, reranker, or fusion rule, and our gains do not come from a stronger embedding model. We certify a *deployment decision*—which existing physical plan to run for a workload window, and when to abstain—under a distribution-free guarantee. Read this way, the closest neighbors fall into four groups, and CWC differs from each in object or granularity rather than in the retrievers it uses: database query/physical-design optimization (different operator substrate and no risk guarantee), per-query routing and selective processing (per-instance, no workload-window guarantee), risk control / conformal methods (per-instance prediction sets rather than a certified plan choice under shift), and hybrid-retrieval operators and benchmarks (the actions we select among, not selectors themselves).

Database query optimization. Classical query optimizers choose physical access paths and operator orders from a cost model (System R’s cost-based access-path selection [7], Volcano’s extensible search [8]), physical-design tools recommend indexes from workload-level what-if analysis [9], and adaptive systems such as Eddies re-route during execution under runtime shifts [10]. WCR follows this tradition but changes the operator substrate: the physical actions are hybrid retrieval plans, not relational joins or scans, and the signals are retrieval telemetry rather than only cardinality estimates.

Workload-level and transductive query optimization. A recent line of database work optimizes plans at the level of a workload rather than a single query. LIMEQO casts offline query optimization as low-rank learning over a workload, exploiting structure shared across queries [11], and Bayesian-optimization planners learn offline plans for repeated workloads [12]. WCR shares this workload-level, transductive stance but on a different substrate (hybrid-retrieval physical plans, not relational latency) and, unlike these methods, attaches a finite-sample guarantee that remains valid under bounded workload shift. On the retrieval side, SEARCHGYM proposes a compositional configuration algebra for hybrid-search orchestration [13]; that work is a benchmarking and reproducibility substrate, whereas WCR is an optimizer that *selects and certifies* plans over such a space under constraints.

Risk control and selective processing for retrieval. Conformal risk control gives distribution-free guarantees, and recent work applies it to ranked retrieval: two-stage conformal risk control bounds retrieval and ranking risk for a single ranker’s output set [14], while selective conformal risk control first selects confident instances and then applies conformal control [15]. Per-query selective processing chooses a search configuration per request, often using query performance prediction, but a recent study shows these per-query gains are small and unreliable across collections [16], [17]. Our certificate differs in granularity and object: it controls the *eval-window* violation rate of a deployed per-cell plan under covariate/workload shift, exploiting the exactly observed cell masses, rather than constructing a per-instance prediction set. This window-level

TABLE I

DESIGN SPACE OF CERTIFIED PLAN SELECTION UNDER SHIFT. EXISTING CERTIFIED-PLAN WORK BOUNDS PER-QUERY LATENCY FOR RELATIONAL SQL; CWC OCCUPIES THE WORKLOAD-WINDOW VIOLATION-RATE CELL FOR HYBRID RETRIEVAL.

	Per-query latency interval	Window violation-rate
Relational SQL	CP4LQO [18]	—
Hybrid retrieval	(per-query CP, unreliable [16])	CWC (ours)

framing is also what lets us avoid the per-query unreliability documented in [16].

Certified plan verification (the closest prior work). The nearest neighbor is CP4LQO [18], which makes learned *relational* query plans verifiable by using conformal prediction to bound each plan’s *per-query latency* before execution, with adaptive-CP handling of distribution shift and a plan search guided by those bounds. We share its goal—a deployable guarantee on a learned plan choice under shift—but differ on three axes that are important to take into account. *Object*: CP4LQO certifies a per-query latency interval; we certify a *workload-window SLO-violation rate* R_Q , the aggregate quantity an SLA or admission controller actually needs, where per-query risk does not accumulate over the window. *Granularity*: CP4LQO reasons per query, so its statistical risk grows with the number of queries; we partition the window into K cells and, because the eval-window cell masses m_c/M are observed exactly (transductive), only the K within-cell expectations carry statistical risk and the mass term is estimation-error-free. *Domain and operation*: CP4LQO targets relational SQL and guides plan *construction*; we target constrained *hybrid retrieval* (BM25/dense/SPLADE/ColBERTv2/fusion/rerank) and make the certificate operational through abstention to a selection-split feasibility-best fallback. Table I places both in a small design space. We position our contribution as a workload-window certified *compiler* for hybrid retrieval, not a new conformal-prediction technique.

Learned query optimization, routing, and policy selection. Learning-augmented optimizers such as Bao steer query optimization while integrating with existing engines [19], and per-query routing / offline policy selection (e.g. contextual-bandit action selection with offline evaluation [20]) choose an action per request from request-local features or logged rewards. WCR learns plan choices from prior executions like these, and we use query-only and scalar-telemetry routers as baselines, but its unit of generalization is a workload cell in retrieval-telemetry space: it observes an unlabeled window, builds reusable cells, and returns a plan frontier and risk report rather than a per-instance relational-cost prediction or contextual classification.

Hybrid-retrieval operators, reranking, and benchmarks. The actions we select among are standard hybrid-retrieval operators: BM25 [1], dense (E5 [2], BGE-M3 [21], Qwen3-Embedding [22], NV-Embed [23]), learned-sparse SPLADE [3], late-interaction ColBERT/ColBERTv2 [4], [24], reciprocal-rank fusion [5], and neural rerankers such as

RankZephyr [6], which trade quality for latency. WCR treats each as a physical action that may or may not be chosen for a cell, rather than proposing a new operator. Finally, the BEIR benchmark systematized the zero-shot evaluation of retrieval models [25]; its metrics, however, score ranking quality alone and say nothing about deployability under a latency SLO; we evaluate constrained score, satisfaction, latency, regret, and rank-gated comparisons so quality and systems feasibility are tested together.

III. PROBLEM AND OPTIMIZER MODEL

Hybrid retrieval exposes a catalog of executable physical actions rather than a single scoring function. *On the word “telemetry”*. We use it for everything the optimizer may read *before* committing a plan for the incoming window, which is of two kinds: (i) quantities computed or estimated up front for the query at hand—query-derived features, filter-survival estimates, and cheap retrieval-probe outputs; and (ii) quantities genuinely observed from execution, such as latency, which are available for the labeled historical window but never for the window being compiled. Compiling the incoming window uses (i) on that window plus (i)–(ii) on history; no measurement of the incoming window’s own execution is ever used. *Notation*: $\mathcal{W}_{\text{tr}}, \mathcal{W}_{\text{ev}}$ are the training and (unlabeled) evaluation workload windows; $\mathcal{A}$ the finite catalog of physical actions; $x(q, \mathcal{W})$ the observable telemetry for query q computed in the context of the window $\mathcal{W} \in \{\mathcal{W}_{\text{tr}}, \mathcal{W}_{\text{ev}}\}$ that contains it (the window enters only through window-level statistics such as filter survival and catalog state); B, F the latency budget and metadata-filter setting; s, ℓ , and $u(q, a; B, F)$ the ranking score, latency, and constrained utility of action a ; and $\mathcal{C}, \gamma, \pi$ and $\mathcal{D}_{\mathcal{W}}$ the cell partition, assignment map, cell-plan binding, and compiled workload design.

Let $\mathcal{W}_{\text{tr}} = \{q_i\}_{i=1}^{n_{\text{tr}}}$ be a labeled training workload and let $\mathcal{W}_{\text{ev}} = \{q_i\}_{i=1}^{n_{\text{ev}}}$ be an unlabeled evaluation workload window. Let $\mathcal{A}$ be a finite catalog of physical actions. An action $a \in \mathcal{A}$ may be BM25 with pre-filtering, BM25 with post-filtering, a dense vector plan, reciprocal-rank fusion, an imported external ranking, or a reranking-depth plan. We use “physical action” consistently for these executable plans.

Each query has an observable telemetry vector $x(q, \mathcal{W}) \in \mathbb{R}^d$. Telemetry can include query-derived features, cheap retrieval-probe statistics, branch overlap, score concentration, filter survival estimates, and latency observations. Evaluation relevance labels are not observable when compiling $\mathcal{W}_{\text{ev}}$.

Definition 1 (Constrained utility and feasibility). For query q , action a , latency budget B , filter setting F , and penalty $\lambda > 0$, the constrained utility is

$$u(q, a; B, F) = s(q, a; F) - \lambda[\ell(q, a; F) - B]_+, \quad (1)$$

where $[z]_+ = \max\{0, z\}$. The feasibility indicator is

$$\text{feas}(q, a; B, F) = \mathbf{1}\{\ell(q, a; F) \leq B\}. \quad (2)$$

For an evaluated set S and policy $\rho : S \rightarrow \mathcal{A}$, satisfaction and regret are

$$\text{Sat}(S, \rho) = \frac{1}{|S|} \sum_{q \in S} \text{feas}(q, \rho(q); B, F), \quad (3)$$

$$\text{Reg}(S, \rho) = \frac{1}{|S|} \sum_{q \in S} \left[\max_{a \in \mathcal{A}} u(q, a; B, F) - u(q, \rho(q); B, F) \right]. \quad (4)$$

These metrics separate systems feasibility from ranking quality: a policy cannot be considered successful if it wins only by violating the latency budget.

Definition 2 (Workload compilation). A workload compiler chooses a partition $\mathcal{C} = \{c_1, \dots, c_K\}$ of $\mathcal{W}_{\text{tr}} \cup \mathcal{W}_{\text{ev}}$, an assignment map $\gamma(q) \in \{1, \dots, K\}$, and a binding $\pi : \mathcal{C} \rightarrow \mathcal{A}$. The deployed policy for an evaluation query is

$$\rho_{\mathcal{C}, \pi}(q) = \pi(c_{\gamma(q)}), \quad q \in \mathcal{W}_{\text{ev}}. \quad (5)$$

The compiler may use relevance labels from $\mathcal{W}_{\text{tr}}$ to bind actions, but it may use only telemetry from $\mathcal{W}_{\text{ev}}$ when assigning evaluation queries to cells. For a fixed partition, the empirical binding objective is

$$\pi^*(c) = \arg \max_{a \in \mathcal{A}} \sum_{q \in c \cap \mathcal{W}_{\text{tr}}} u(q, a; B, F). \quad (6)$$

If a cell contains no training queries, the implementation falls back to the globally best training action under the same utility.

Definition 3 (Compiled workload design). The compiler returns

$$\mathcal{D}_{\mathcal{W}} = (\mathcal{C}, \mathcal{P}, \mathcal{R}, \mathcal{G}). \quad (7)$$

Here $\mathcal{C}$ is the workload-cell partition. $\mathcal{P}$ records, for each cell, the empirical plan frontier over ranking score, latency, and constrained utility. $\mathcal{R}$ records the estimated response of alternative actions in the cell. $\mathcal{G}$ records empirical stability statistics such as plan margin, bootstrap support, and risk flags (distinct from the distribution-free certificate of Section V).

Assumption 1 (Workload-window observability). The system observes $\mathcal{W}_{\text{ev}}$ and can compute telemetry $x(q, \mathcal{W}_{\text{tr}} \cup \mathcal{W}_{\text{ev}})$ before final plan execution. It does not observe evaluation relevance labels during compilation.

Proposition 1 (Cell-wise empirical optimality). For any fixed partition $\mathcal{C}$ and any cell c with at least one training query, the binding rule in Eq. (6) maximizes empirical constrained utility among all policies that assign one constant action to every training query in c .

Proof. For fixed $\mathcal{C}$ and c , a cell-constant policy is fully determined by its chosen action $a \in \mathcal{A}$. Eq. (6) enumerates the finite action catalog and selects the action with the largest empirical utility sum on $c \cap \mathcal{W}_{\text{tr}}$. No other cell-constant action can have a larger empirical objective value by definition of the argmax. $\square$

This proposition is intentionally modest. It does not claim optimality over all possible partitions or future workloads. It

states the optimizer invariant used by WCR once a workload partition is fixed. The empirical question in the rest of the paper is whether a telemetry-derived partition gives useful cell-constant physical designs under the constrained objective.

IV. CWC: WORKLOAD-WINDOW PLAN COMPILER

This section describes the compiler core of CWC (its unguarded form WCR); Section V adds the abstain rule and certificate that complete CWC. It is deliberately simple: the paper tests whether workload-level physical-plan compilation carries signal, not whether a complex controller can overfit a small benchmark. The implementation has four stages, shown in Figure 1: construct an action catalog, compute workload telemetry, partition the workload, and bind/certify cell actions.

A. Physical Action Catalog and Telemetry

The action catalog treats retrieval systems as executable physical actions. In the main external-operator gate, $\mathcal{A}$ includes BM25 pre-filtering and post-filtering, dense retrieval, E5-base dense retrieval [2], reciprocal-rank fusion [5], weighted fusion, rerank depth 20, and external rankings imported from SPLADE [3], ColBERTv2 [4], and Qwen3-Embedding-0.6B [22]. Static use of each action is also evaluated as a baseline.

The full telemetry vector contains four feature families: query/constraint features (query length, digit ratio, filter selectivity, latency budget); candidate-set features (candidate count); lexical and dense probe features (BM25 and dense top score, entropy, and gap); and interaction/cost features (branch overlap, probe latency). Feature ablations (Section VII) remove these families to test that the compiler uses retrieval telemetry, not only query metadata.

B. Cell Construction

Per block, WCR computes one telemetry vector $z(q)$ per query, standardizes them using the combined-window mean and variance, and runs k-means (K clusters, 20 restarts, block seed). The main result uses $K=6$; robustness runs evaluate $K=5$, a k -sweep, and two unsupervised selectors (an elbow selector at maximum distance from the log-inertia chord, and the standard silhouette score). Using evaluation telemetry here is a workload-aware systems choice—the compiler observes the current window before execution, as an optimizer observes a query before choosing a plan—but it never observes evaluation relevance labels.

We distinguish two regimes that differ only in this clustering/binding step. The exploratory WCR used in the pilot and audit (Sections VII-F, VII-H) clusters on the combined train+evaluation window and binds on the same training rows; it makes optimizer-level claims only and does *not* invoke the certificate, because Assumption 2 (calibration independence) is not satisfied. The certified CWC of Section V instead uses an independent selection/calibration split, which is what licenses Proposition 2; all certificate results (Sections VII-B–VII-C) and the canonical rank gate (Section VII-A) use this split.

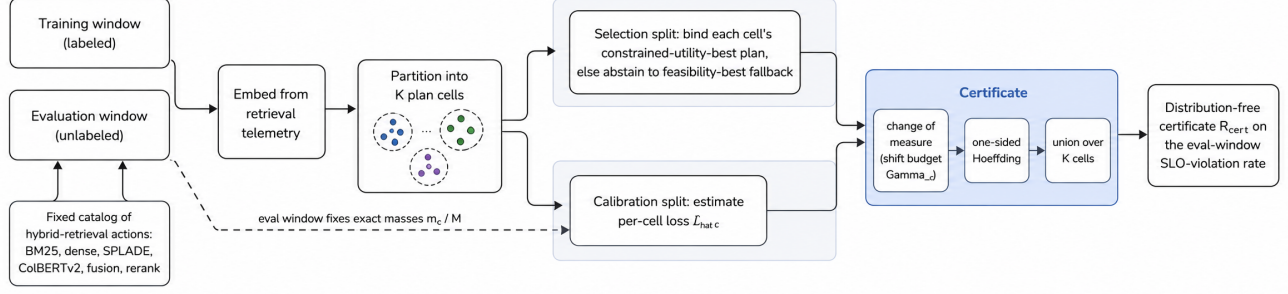


Fig. 1. The CWC pipeline. An unlabeled evaluation window and a labeled training window over a fixed action catalog are embedded into K plan cells; the *selection* split binds each cell’s plan (or abstains to a feasibility-best fallback), the *calibration* split estimates $\hat{L}_c$, and the eval window fixes the masses m_c/M . A change of measure under Γ_c with one-sided Hoeffding and a union over K cells gives the distribution-free certificate R^{cert} on the eval-window SLO-violation rate.

C. Cell Binding and Fallback

For each cell, WCR selects the action with the highest mean training constrained utility among training queries assigned to that cell. Ties are resolved deterministically by the sorted action order. If a cell has no training queries, WCR falls back to the globally best training action under the same scope. This fallback is important for reproducibility because transductive clustering can create evaluation-only cells.

Algorithm 1 gives the complete procedure.

D. Plan Frontier and Empirical Stability Report

The plan frontier is an empirical audit object, not a second optimizer: per cell, WCR records each action’s mean constrained utility, satisfaction, and latency on the cell’s training queries, the selected-plan margin (gap to the second-best action), and the frontier width (actions within $\epsilon=0.01$ of the best), identifying cells where several plans are close. The empirical stability report $\mathcal{G}$ is a diagnostic, distinct from the distribution-free *certificate* of Section V (for which we reserve the word “certificate”); it is not a correctness proof or a promise of future optimality. It reports bootstrap support (fraction of $B_{\text{boot}}=200$ resampled train-cell workloads reselecting the deployed action), the margin, and a risky flag (support $< \tau_s=0.70$ or margin $< \tau_m=0.02$). These thresholds feed only the stability table; the main Rank-1 decision does not tune them.

The response surface $\mathcal{R}$ stores counterfactual action responses for diagnosis; we do not cluster on it directly (pure counterfactual clustering failed the Rank-1 gate in our negative variants), keeping the telemetry compiler as the engine while still exposing counterfactual information.

E. Why This Is Not a Router

WCR differs from a per-query router in three ways: its observable unit is a workload window rather than an isolated query; its decision object is a cell-level physical design $\mathcal{D}_{\mathcal{W}} = (\mathcal{C}, \mathcal{P}, \mathcal{R}, \mathcal{G})$ that reuses plans over stable workload regions instead of deciding each query independently; and it returns frontier and stability flags alongside the deployed

Algorithm 1 WCR: risk-aware workload compilation (the exploratory regime). The certified variant CWC replaces the single train set used in lines 7–8 by the selection/calibration split of Section VI so that Assumption 2(iii) holds; all other steps are unchanged.

Require: $\mathcal{W}_{\text{tr}}, \mathcal{W}_{\text{ev}}$, action catalog $\mathcal{A}$, budget B , filter F , feature map x , cell count or selector K , bootstrap iterations B_{boot}

- 1: Compute query telemetry $z(q) = x(q, \mathcal{W}_{\text{tr}} \cup \mathcal{W}_{\text{ev}})$ for all train and evaluation queries
- 2: Standardize all telemetry vectors in the workload window
- 3: Cluster train and evaluation telemetry into cells $\mathcal{C} = \{c_1, \dots, c_K\}$
- 4: Let a_{global} be the best training action by mean constrained utility
- 5: **for** each cell $c \in \mathcal{C}$ **do**
- 6: **if** $c \cap \mathcal{W}_{\text{tr}} \neq \emptyset$ **then**
- 7: $\pi(c) \leftarrow \arg \max_{a \in \mathcal{A}} \sum_{q \in c \cap \mathcal{W}_{\text{tr}}} u(q, a; B, F)$
- 8: **else**
- 9: $\pi(c) \leftarrow a_{\text{global}}$
- 10: **end if**
- 11: Compute frontier statistics $\mathcal{P}(c)$ and action responses $\mathcal{R}(c)$
- 12: Bootstrap $c \cap \mathcal{W}_{\text{tr}}$ for B_{boot} iterations to compute stability report $\mathcal{G}(c)$
- 13: **end for**
- 14: **for** each $q \in \mathcal{W}_{\text{ev}}$ **do**
- 15: Deploy $\rho_{\mathcal{C}, \pi}(q) = \pi(c_{\gamma(q)})$
- 16: **end for**
- 17: **return** $\mathcal{D}_{\mathcal{W}} = (\mathcal{C}, \mathcal{P}, \mathcal{R}, \mathcal{G})$ and deployed actions

actions. These properties make it closer to a physical-plan compiler than to a classifier over retriever names.

The current implementation uses k-means and empirical cell binding as one instantiation; the broader abstraction is the compiled design object, in which alternative partitioners

or binding rules could be substituted while preserving the workload-level optimizer contract.

V. A CERTIFICATE FOR WORKLOAD-WINDOW PLAN SELECTION UNDER SHIFT

The compiler of Section IV reports *empirical* stability statistics. We now strengthen this into a *distribution-free, finite-sample guarantee* on the eval-window constraint-violation rate that holds even when the eval workload window is shifted relative to training. This is the property that static and per-query baselines do not provide, and it is the formal contribution of CWC (Certified Window Compiler), the guarded variant of the compiler.

Setup. Let the deployed loss be the SLO/constraint-violation indicator $L(q, a) = \mathbf{1}\{\ell(q, a) > B\} \in [0, 1]$ for budget B . A measurable map $\varphi : \mathcal{X} \rightarrow \{1, \dots, K\}$ assigns each query to one of K plan cells; cell c deploys plan π_c . Write P_c and Q_c for the train- and eval-window cell-conditional query laws, $\bar{L}_c$ for the empirical loss of π_c on n_c calibration queries in cell c , $\bar{L}_c = \mathbb{E}_{P_c}[L(\cdot, \pi_c)]$, and m_c/M for the (observed) eval-window mass of cell c . We adopt a finite-population model: P_c and Q_c are the empirical laws of cell c over a finite train and eval universe, the density ratio w_c is the ratio of these empirical cell-conditional masses (finite and bounded by Γ_c when P_c puts positive mass on every eval atom, Assumption 2(ii)), and the certified target $R_Q = \sum_c (m_c/M) \mathbb{E}_{Q_c}[L(\cdot, \pi_c)]$ is the realized finite eval-window violation rate the experiments measure. The n_c calibration losses are sampled from P_c (the only randomness in Proposition 2); under this model the change-of-measure and Hoeffding steps below are exact for the realized window rather than for a separate population.

Assumption 2 (Bounded loss, bounded shift, calibration independence). (i) $L \in [0, 1]$. (ii) For each cell the eval law is absolutely continuous w.r.t. the train law with a bounded density ratio $\|dQ_c/dP_c\|_\infty \leq \Gamma_c < \infty$. (iii) φ and the plan choice $\{\pi_c\}$ are computed from an independent selection split and the unlabeled eval features only, so that conditional on $(\varphi, \{\pi_c\})$ the n_c calibration queries in cell c are i.i.d. draws from P_c .

Proposition 2 (Workload-window shift certificate). Under Assumption 2, with probability at least $1 - \delta$ over the calibration sample,

$$R_Q \leq R^{\text{cert}} := \sum_{c=1}^K \frac{m_c}{M} \min \left\{ \Gamma_c \left(\hat{L}_c + \sqrt{\frac{\log(K/\delta)}{2n_c}} \right), 1 \right\}. \quad (8)$$

Both the fallback plan and the per-cell abstain decision are functions of the *selection* split alone: the fallback is the selection-split feasibility-best action, and a cell abstains when its selection-split support/margin fall below the fixed thresholds of Section IV-D. The deployed plan π_c (either the bound action or the fallback) is therefore fixed before calibration, the calibration split certifies only that one deployed plan per cell, and there are exactly K Hoeffding events—so no union bound over candidate plans is introduced and (8) holds as

stated with $\log(K/\delta)$ for whichever plan is deployed. (We do not claim abstention monotonically lowers R^{cert} or the realized rate—the fallback has its own nonzero loss; we claim only that validity is preserved because the deployed plan is fixed before calibration.)

Proof. Conditioning on the realized eval window fixes the masses m_c/M . For each cell, a change of measure under Assumption 2(i)–(ii) gives $\mathbb{E}_{Q_c}[L] = \mathbb{E}_{P_c}[w_c L] \leq \Gamma_c \bar{L}_c$ since $L \geq 0$ and $w_c \leq \Gamma_c$ pointwise. By Assumption 2(ii), π_c is fixed given the selection split and the n_c calibration losses are i.i.d. in $[0, 1]$, so one-sided Hoeffding yields $\bar{L}_c \leq \hat{L}_c + \sqrt{\log(K/\delta)/(2n_c)}$ with probability $1 - \delta/K$. A union bound over the K cells and the trivial cap $\mathbb{E}_{Q_c}[L] \leq 1$ give (8). Whether π_c is the bound action or the fallback is decided on the selection split, so the deployed plan is fixed before calibration and (8) certifies that one plan per cell; we make no claim that abstention reduces R^{cert} . *Boundary cases:* a cell with $n_c=0$ takes the trivial cap 1 and abstains by construction; if $\hat{L}_c=0$ its term is $\min\{\Gamma_c \sqrt{\log(K/\delta)/(2n_c)}, 1\} > 0$ (why the bound stays loose at small n_c); empty eval cells ($m_c=0$) contribute zero mass. The variance-adaptive (empirical-Bernstein) form replaces the Hoeffding radius by the Maurer–Pontil radius $\sqrt{2\hat{\sigma}_c^2 \log(2K/\delta)/n_c + 7 \log(2K/\delta)/(3(n_c - 1))}$, with $\hat{\sigma}_c^2$ the empirical loss variance in cell c ; it holds by the same union bound and tightens R^{cert} from ≈ 0.30 to ≈ 0.26 at scale. $\square$

Reading the certificate. Three points matter for deployment. First, the guarantee is at the *workload-window* granularity: the eval masses m_c/M are observed exactly (the transductive advantage), so only the K within-cell expectations carry statistical risk. This sidesteps the documented unreliability of per-query selective processing [16]. Second, the bound is *operational*: it turns the empirical stability report into a deployment rule via the selection-split abstain mechanism; abstention preserves validity because the deployed plan is fixed before calibration (it is not claimed to monotonically reduce R^{cert} or the realized rate). Third, the dominant term is $\Gamma_c \sqrt{\log(K/\delta)/(2n_c)}$, so the certificate is tight only when per-cell calibration counts n_c are large; this directly motivates choosing K to keep cells well-populated rather than to maximize a separation score, and it predicts the loose bounds we observe on a small benchmark (Section VII).

Certification contract. We use three labels consistently for the rest of the paper, to keep theorem-validity, empirical upper-bounding, and the model-based variant strictly separate.

- **THEOREM-VALID.** Proposition 2 holds, distribution-free and finite-sample, *conditional on* Γ_c being a true upper bound on the within-cell density ratio (Assumption 2(ii)).
- **HEADLINE PLUG-IN.** The reported R^{cert} substitutes a conservative estimate of Γ_c from a regularized within-cell density-ratio classifier. We do not call this “valid”; we state only the operational fact that *the plug-in bound upper-bounds the realized violation in every evaluated run*, under the assumption that the estimated Γ_c is a valid

Algorithm 2 CWC: certified workload-window compilation. Steps 1–5 use only the selection split and unlabeled eval features; step 6 is the sole use of calibration losses.

Require: selection split S , calibration split C (disjoint, labeled), unlabeled eval window E , catalog $\mathcal{A}$, budget B , cells K , level δ

- 1: fit standardizer and K -means φ on telemetry of $S \cup E$ (no labels)
- 2: bind each cell’s plan $\pi_c \leftarrow \arg \max$ mean constrained utility on $S \cap c$
- 3: compute selection-split support/margin; mark fragile cells
- 4: fallback $\pi_c^{\text{fb}} \leftarrow$ feasibility-best action on $S \cap c$; deploy π_c^{fb} on fragile cells
- 5: estimate Γ_c from a within-cell S -vs- E density-ratio classifier; observe masses m_c/M on E
- 6: **(calibration)** $\hat{L}_c \leftarrow$ mean loss of the *deployed* plan on $C \cap c$
- 7: **return** deployed policy and R^{cert} via (8)

bounded-shift budget. It is not an end-to-end finite-sample Γ_c certificate.

- **MODEL-BASED UCB.** Remark 1 replaces Γ_c by a data-driven upper confidence bound $\bar{\Gamma}_c$ that is valid *under the stated log-linear density-ratio model* (not model-free); it removes the plug-in step but is numerically vacuous at our sample sizes, and is never the source of a headline number.

We additionally report robustness to Γ_c misspecification; misspecification and small n_c loosen the bound but, under THEOREM-VALID, do not affect its validity.

Algorithm 2 makes the certified data flow explicit: every object that the certificate conditions on is frozen on the selection split (plus unlabeled eval telemetry) before any calibration loss is read.

Proposition 3 (Transductive width advantage over per-query certification). Fix the per-cell certified terms $U_c = \min\{\Gamma_c(\hat{L}_c + \varepsilon_c), 1\}$. The window certificate weights them by the *observed* masses m_c/M , contributing no weight-deviation term. A certifier in the *distributional class*—one whose guarantee holds for the eval law Q rather than the realized window, and which therefore does not condition on $\{m_c/M\}$ (the class of per-query conformal latency certification, e.g. [18])—bounds R_Q by combining the same U_c with cell weights it has not observed; preserving validity over all admissible compositions then costs an additive penalty of at least

$$\Delta_{\text{mass}} \geq c_0 \sum_c U_c \sqrt{\frac{q_c(1-q_c) \log(1/\delta)}{M}} > 0$$

for a universal $c_0 > 0$, whenever some cell has $q_c \in (0, 1)$ and $U_c > 0$. The penalty is governed by the window size M , not the calibration size n_c , so it does *not* vanish as $n_c \rightarrow \infty$. This is a separation between guarantee *targets* (realized window vs. eval law), not an impossibility against all algorithms (a scheme

permitted to read $\{m_c\}$ is, by definition, a transductive window certifier).

Proof. Both classes deploy a fixed per-cell plan and obtain the same U_c valid w.p. $1 - \delta/K$ each (A1–A3, one-sided Hoeffding, union over K); they differ only in the cell weights used to bound $R_Q = \sum_c (m_c/M) \mathbb{E}_{Q_c}[L]$. Conditioning on the realized window makes m_c/M constants, so $\sum_c (m_c/M) U_c$ is valid with *no* weight-deviation term. A distributional certifier does not observe $\{m_c\}$; using weights $\hat{q}_c$ it must satisfy $\sum_c \hat{q}_c U_c \geq \sum_c q_c \mathbb{E}_{Q_c}[L]$ uniformly over admissible q , forcing $\hat{q}_c$ to a one-sided $1 - \delta$ upper envelope of q_c . Since $m_c \sim \text{Binomial}(M, q_c)$, a two-point (Bretagnolle–Huber) argument shows no such envelope is smaller than $c_0 \sqrt{q_c(1-q_c) \log(1/\delta)/M}$; summing against U_c gives Δ_{mass} . The penalty depends on M , not n_c , so it persists as $n_c \rightarrow \infty$. (A certifier avoiding mass estimation by a per-query bound uniform over the M eval queries instead carries $\log(M/\delta')$ rather than $\log(K/\delta')$ in its radius—the same-signed penalty for $M > K$.) $\square$

The separation is between guarantee *targets*: a scheme handed the realized masses is, by definition, a transductive window certifier (i.e. CWC), not an information-theoretic impossibility against all algorithms. In our regime ($K=4$, $M \approx 860$, $\Gamma_c \approx 3$) the transductive design buys ≈ 0.02 of bound width via the mass-deviation route and ≈ 0.12 via the per-query union-bound route ($\log M$ vs. $\log K$); the constant c_0 is not optimized, only its positivity and n_c -independence are used.

Remark 1 (Certifying the shift budget end-to-end). The bound (8) treats Γ_c as given. Under a log-linear density-ratio model $dQ_c/dP_c \propto \exp(\beta_c^\top \phi)$ with bounded features ($\|\phi\| \leq R_c$), the within-cell train-vs-eval classifier yields $\Gamma_c \leq \exp(R_c \|\beta_c\|)$, and a confidence set for β_c gives a data-driven upper bound $\bar{\Gamma}_c(\delta') \geq \Gamma_c$ with probability $1 - \delta'$. Splitting the budget ($\delta/2$ to the Hoeffding events, $\delta/2K$ per-cell to the $\bar{\Gamma}_c$ events) replaces Γ_c by $\bar{\Gamma}_c$ in (8) and removes the assumed-shift-magnitude caveat, giving a guarantee valid end-to-end. We verify this variant is valid in every evaluated run; however, on a 100-query collection it is numerically vacuous, because certifying Γ_c from $n_c \approx 20$ –50 points inflates $\bar{\Gamma}_c$ until the cap dominates. Closing the validity gap and obtaining a *tight* end-to-end bound therefore both require larger workload windows.

VI. EXPERIMENTAL PROTOCOL

The evaluation is designed around a hard decision rule: the candidate must be statistically rank-first over complete workload blocks. This avoids selecting a method that wins one metric but loses the ranking comparison after accounting for budgets and filter conditions.

A. Workloads and Blocks

The *pilot* gate (our earliest study; canonical evidence is the multi-dataset gate of Section VII-A) uses a cached 100-query NFCorpus workload from BEIR [25]. Metadata filters are

TABLE II

ACTION CATALOG AND ACCOUNTING. “MEASURED” MEANS PRODUCED BY THE LOCAL PILOT; “IMPORTED” MEANS RANKING ORDER IS IMPORTED AND EVALUATED IN THE SHARED CONSTRAINED OBJECTIVE.

Action family	Source	Accounting
BM25 pre/post	local	measured ranking/latency
Dense/E5	local	measured ranking/latency
RRF/weighted fusion	local	measured derived plan
Rerank depth 20	local	measured pilot action
SPLADE/Qwen/ColBERT	imported	shared objective rows
Cross-dataset actions	local	shared pilot rows

deterministic recency and document-length buckets. A block is a tuple

$$b = (\text{dataset}, B, F, \text{seed}), \quad (9)$$

where $B \in \{40, 80\}$ ms is the latency budget and F is a selectivity setting in $\{1.0, 0.5, 0.1\}$. For each seed, the workload qids are split into 50% training and 50% evaluation windows. The pilot and audit use this *four-factor* block schema (dataset, B , F , seed); the pilot uses seeds $\{13, 17, 23, 29, 31\}$, yielding 30 complete blocks. The canonical study of Section VII-A instead uses a *three-factor* block (dataset, B , seed) with the three selectivities folded into each block as workload-item strata, so its block count is 3 datasets $\times$ 3 budgets $\times$ 5 seeds = 45; this is why selectivity appears as a block dimension in the pilot but as item strata in the canonical study.

We separately run a cross-dataset local-catalog audit over SciFact, NRCorpus, and FiQA. This audit uses the shared cached pilot action rows rather than the external Qwen/SPLADE/ColBERT imports. It spans the same selectivities and budgets with 20 random seeds, yielding 360 complete blocks. Because the action catalog differs from the main external-operator gate, we report this audit as robustness evidence rather than as the primary claim.

B. Action Catalog and Baselines

Table II summarizes the physical actions. All methods in a block read from the same cached action rows. For imported rankings, the ranking order is imported from the external model and then evaluated under the same filter and latency accounting used by the pilot framework. The claim is therefore policy quality under a fixed catalog, not an end-to-end systems benchmark of every external model’s serving stack.

This distinction is central to the interpretation of the results. A fully executable serving benchmark would include model-specific inference, indexing, hardware, batching, and cache behavior for each external system. Our experiment instead asks whether a workload compiler can choose among already materialized physical actions under a common constrained objective. This isolates policy selection from model-serving engineering.

The candidate is WCR with workload cells and cell-level plan binding. The main configuration uses $K = 6$ and the full telemetry feature set. Baselines include static execution of one physical action for the whole workload and four

adaptive or calibrated selectors, named by the information each is allowed to use: *QueryOnly-RF*, a random forest (RF) trained to pick the action from query-side features only; *ScalarTelemetry-RF*, the same random forest given a scalar summary of the retrieval telemetry instead of the full feature set; *CostGreedy-cal*, which picks the highest-nDCG action whose calibrated (“-cal”) training-window $p95$ latency fits the budget; and *StaticBest-cal*, the single action with the best calibrated training-window constrained utility. Throughout, “SOTA” denotes the four strong published retrieval operators of Table II used as static plans. Control variants include train-only compilation, query-only cell construction, metadata-only features, feature-family removals, and random cells.

C. Metrics, Ranking, and Tests

For each method m and block b , we compute the mean constrained score over evaluation queries, denoted $S_{m,b}$. Methods are ranked within each block by $S_{m,b}$, with lower rank better. The average-rank statistic is

$$\bar{r}_m = \frac{1}{|\mathcal{B}|} \sum_{b \in \mathcal{B}} \text{rank}_b(m). \quad (10)$$

We use two evaluation criteria, deliberately kept separate. The *pilot Rank-1 gate* (Section VII-F) is a strict screen requiring WCR to have the smallest $\bar{r}_m$ and to be significantly better-ranked than *each* baseline after Holm correction; it is a go/no-go filter on the early single-collection study, not the headline claim. The *canonical evaluation* (Section VII-A) reports two families instead of a single pass/fail: a *SOTA-static rank gate* (is CWC rank-first and Holm-significant against the strong static operators?) and a *matched calibration-aware parity test* (is CWC at least tied, on the same paired blocks, with the calibration-aware selectors that share its violation-avoidance mechanism?). We do not claim significance over every baseline in the canonical study, because CWC is by design tied with the calibration-aware selectors on mean utility and differentiated by the certificate.

Pairwise tests use the complete block as the paired unit. For each baseline m , the tested samples are the per-block rank differences $\text{rank}_b(m) - \text{rank}_b(\text{WCR})$. The null is that the median paired rank advantage is non-positive. We apply Holm correction over the family of candidate-versus-baseline comparisons in the same gate. In addition to rank, we report mean constrained score, satisfaction, latency, and regret as defined in Section III.

D. Certified Split Protocol

The certificate of Proposition 2 requires that cell assignment and plan choice be independent of the calibration losses (Assumption 2(iii)). The certified runs (CWC; the canonical 45-block gate of Section VII-A and all certificate experiments) therefore use an explicit split of the labeled training window into two disjoint parts: a *selection* split, used to fit the cell partition φ , to bind each cell’s plan π_c , and—together with the unlabeled eval telemetry—to estimate the shift budget Γ_c (the within-cell selection-vs-eval density-ratio classifier

of Algorithm 2, step 5); and a *calibration* split, used only to estimate $\hat{L}_c$. Unlabeled evaluation telemetry may inform clustering and fixes the cell masses m_c/M , but evaluation relevance labels are never used, and no calibration loss enters plan selection or Γ_c . The exploratory WCR of the pilot and audit (Sections VII-F, VII-H) omits this split—it clusters on the combined window and binds on the same rows—so it makes optimizer-level claims only and does not invoke the certificate.

E. Reproducibility Notes

Environment. All experiments run on a single workstation—an Intel Core i9-9900K (8C/16T), 32 GB RAM, and one NVIDIA RTX 4090 (24 GB)—under Ubuntu 20.04 with PyTorch 2.10 (CUDA 12.8), scikit-learn, HNSWlib, and Sentence-Transformers (cross-encoder ms-marco-MiniLM-L-6-v2, dense encoder E5-base).

Concrete settings used throughout: nDCG@10 quality metric; constrained-utility penalty (Definition 1) $\lambda = 0.15$ per unit of normalized budget overrun; latency taken from the cached action rows (ms). Imported external rankings receive the same per-query latency/cost accounting as locally measured actions, so all methods in a block are compared on identical bookkeeping rather than each model’s production serving stack. The main gate imports true rankings for Qwen3-Embedding-0.6B, SPLADE, and ColBERTv2 (NV-Embed [23] and RankZephyr [6] are discussed but not executed as actions, their outputs being unavailable under our constraints); each result directory records action rows, seeds, budgets, selectivities, feature sets, k -selection rules, and rank-gate inputs.

VII. RESULTS AND ANALYSIS

Reading guide and canonical result. We lead with the canonical result and the certificate, then demote corroborating studies. The headline is the 45-block multi-dataset rank gate (Section VII-A; 3 datasets, 4 static + 2 calibration-aware baselines), where CWC ranks first and beats all four SOTA operators while tying the two calibration-aware selectors. The certificate (Sections VII-B–VII-C) then upper-bounds the realized SLO-violation rate and tightens with scale, and the SLA admission-control study (Section VII-D) reuses the same 45 blocks and pool. A single-collection NFCorpus pilot (Section VII-F, 30 blocks) and a 360-block cross-dataset audit (Section VII-H) close the section as corroborating robustness, not the headline. Throughout, CWC denotes WCR augmented with the selection-split abstain rule of Section V; when no cell abstains, $CWC \equiv WCR$, so the un-guarded WCR numbers double as an ablation of the abstain mechanism.

A. Average-rank comparison against SOTA operators

We open with the headline utility result, a standard average-rank test on the constrained objective; the certificate that backs these deployment decisions follows in Sections VII-B–VII-C. Over 45 complete blocks (three datasets, three budgets at 80–160 ms, five seeds) under the bounded workload shift, we rank every method per block by mean constrained

TABLE III
AVERAGE RANK (LOWER IS BETTER) OVER 45 BLOCKS ON SciFact+FiQA+NFCORPUS WITH REAL EXTERNAL OPERATORS. CWC IS RANK-1 AND SIGNIFICANTLY BEATS ALL FOUR SOTA OPERATORS UNDER A ONE-SIDED HOLM-CORRECTED WILCOXON TEST; IT IS STATISTICALLY TIED WITH THE TWO CALIBRATION-AWARE SELECTORS. “WIN BLOCKS” IS THE NUMBER OF BLOCKS (OUT OF 45) IN WHICH CWC HAS STRICTLY HIGHER MEAN CONSTRAINED UTILITY THAN THE BASELINE IN THAT ROW.

Method	Avg. rank	Holm p vs. CWC	win blocks
CWC (certified)	2.511	–	–
Static-SPLADE (SOTA)	3.067	1.5×10^{-2}	26/45
Static-Qwen3 (SOTA)	3.511	8.1×10^{-4}	25/45
StaticBest-cal	3.622	2.8×10^{-1} (n.s.)	15/45
CostGreedy-cal	3.622	1.1×10^{-1} (n.s.)	19/45
Static-E5 (SOTA)	4.667	1.9×10^{-8}	42/45
Static-ColBERTv2 (SOTA)	7.000	1.7×10^{-13}	45/45

utility (nDCG when SLO-feasible, penalty otherwise) and compare CWC against four SOTA retrieval operators (Qwen3-Embedding, SPLADE, ColBERTv2, E5) and two calibration-aware selectors. Table III reports the average rank of every method in the pool; a Friedman test rejects equality ($\chi^2=135.4$, $p = 9.3 \times 10^{-27}$). For self-containedness, the full configuration (reproduced from the merged action rows, no value hand-set) is: SciFact+FiQA+NFCorpus (1,271 queries, 2,585 workload items) with real Qwen3-Embedding-0.6B, SPLADE, and ColBERTv2; the 45 blocks are 3 datasets $\times$ budgets {80, 120, 160} ms $\times$ seeds {13, 17, 23, 29, 31}, with selectivities {1.0, 0.5, 0.1} as item strata and a selectivity-biased train/eval coin ($p_{\text{train}}=0.8/0.5/0.2$); $K=4$ main cells (sweep {3, 4, 6, 8, 12}), a 50/50 selection/calibration split, the constrained-utility objective (nDCG if feasible, else penalty), and the seven methods listed above tested by per-block rank with Friedman + one-sided Holm–Wilcoxon.

CWC attains average rank 2.511, first among all seven methods, and we read this conservatively. Against the methodologically matched competitors—the two calibration-aware selectors, which also avoid SLO violations by construction—CWC is *statistically tied in blockwise rank under the constrained-utility objective* (one-sided Holm–Wilcoxon on per-block rank differences, $p=0.28$ and $p=0.11$), consistent with the audit of Section VII-H; the defensible claim is *blockwise-rank parity plus an auditable guarantee*, not a tested utility-value win over comparable selectors. Against the four strong static retrievers used directly CWC is Holm-significantly better (winning 25–45 of 45 blocks), but only as a sanity check that a strong retriever used statically is not a substitute for certified selection. The genuine differentiator over the matched selectors is the certificate of Section V: the same utility with a distribution-free guarantee none of the baselines provides, at $\approx$ two orders of magnitude fewer realized violations. Both numbers come from one stated setting—the combined SciFact+FiQA+NFCorpus workload at the binding 80 ms budget with $K=4$, averaged over the five window-assignment seeds—and are the viol. CWC and viol. StaticBest-cal entries of Table IV (0.001 vs. 0.112); no other configuration is pooled into them.

B. Certified behavior under bounded workload shift

We now test the certificate of Proposition 2 and the guarded compiler CWC. We induce a bounded covariate shift between the train and eval windows on the NFCorpus action rows by assigning each (query, selectivity) item to the train or eval window with a selectivity-biased coin: low-selectivity (fast) items skew toward training and high-selectivity (slow) items skew toward evaluation. Both windows therefore contain all strata in different proportions, so the windows overlap and the per-cell density ratio is bounded; a hard partition would instead produce a singular shift for which no method can certify and the bound correctly returns the trivial value one. The loss is the SLO-violation indicator $L = 1\{\ell > B\}$. We use $K=3$ cells, $\delta=0.1$, sample-split selection/calibration, a within-cell density-ratio estimate for Γ_c , and report means over five window-assignment seeds; R^{cert} is the certified bound of (8).

On this NFCorpus precursor ($K=3$, mean over 5 seeds; $R^{\text{cert}}=0.951$ at all budgets) the run reports three findings. First, *the plug-in bound empirically upper-bounds the realized violation in every run*: across all 60 evaluated runs (cell counts $K \in \{3, 4\}$, five seeds, three budgets, including a $1.5 \times \Gamma_c$ -misspecification stress test) the certified bound exceeds the realized eval-window violation, with minimum slack above 0.43. We reserve “theorem-valid” for a true Γ_c ; the headline bound uses a plug-in Γ_c (empirically upper-bounding here), and only the UCB- ρ variant of Remark 1 is end-to-end. No static or per-query baseline supplies any such guarantee. Second, *the guard controls violation under shift*: at the binding budgets CWC lowers the realized SLO-violation rate relative to the strongest calibration-aware static selector (e.g. 0.361 vs. 0.427 at 80 ms and 0.020 vs. 0.036 at 120 ms) at comparable nDCG (0.40 vs. 0.40 and 0.412 vs. 0.417 respectively), by abstaining to the selection-split feasibility-best fallback on 13–32% of eval mass. At the loose budget (160 ms) the workload is easy for all policies and no abstention is needed. Third, and consistent with the theory, *the finite-sample bound is loose on this small benchmark* ($R^{\text{cert}} \approx 0.95$): the dominant term $\Gamma_c \sqrt{\log(K/\delta)/(2n_c)}$ is large because per-cell calibration counts are only $n_c \approx 18$ –50 and the estimated within-cell shift budget is $\Gamma_c \approx 4$ –6. The bound becomes non-vacuous under milder shift, and the analysis predicts that larger workload windows are what tighten it. We therefore claim a *distribution-free and operational workload-window guarantee*, not a tight numerical bound on a 100-query collection.

C. Scaling to a larger multi-dataset workload

To test the prediction that larger workload windows tighten the certificate, we generate action rows on a combined workload of SciFact, FiQA, and NFCorpus using the same real external operators (Qwen3-Embedding, SPLADE, ColBERTv2). After retaining queries with complete action rows across all operators, this yields 1,271 queries and 2,585 workload items (a workload item is a query at a given selectivity), roughly $26 \times$ the single-collection pilot, raising per-cell calibration counts to $n_c \approx 100$ –430. Table IV reports the same certified-shift protocol over $K \in \{3, 4, 6, 8, 12\}$.

TABLE IV

CERTIFIED COMPILER ON THE COMBINED SciFact+FiQA+NFCORPUS WORKLOAD (1,271 QUERIES, REAL EXTERNAL OPERATORS, MEAN OVER 5 SEEDS, BUDGET 80 MS). R^{cert} IS THE CERTIFIED BOUND; VIOLATION IS THE REALIZED EVAL-WINDOW SLO-VIOLATION RATE. THE PLUG-IN BOUND EXCEEDS THE REALIZED VIOLATION IN EVERY RUN OF BOTH THE 25 BUDGET-80 K-SWEEP RUNS (SHOWN) AND THE 15 MAIN- $K=4$ RUNS ACROSS THE THREE BUDGETS.

K	R^{cert}	viol. CWC	viol. StaticBest-cal	nDCG CWC	nDCG static
3	0.358	0.001	0.112	0.589	0.636
4	0.298	0.001	0.112	0.609	0.636
6	0.440	0.001	0.112	0.602	0.636
8	0.518	0.002	0.112	0.602	0.636
12	0.655	0.002	0.112	0.602	0.636

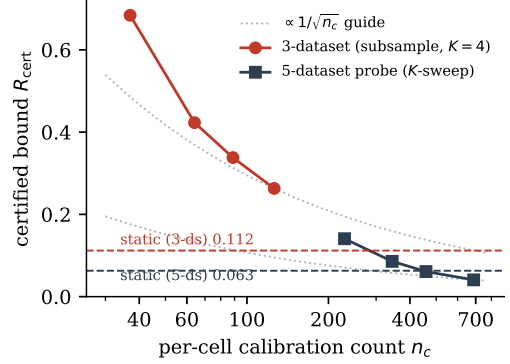


Fig. 2. Certificate tightening with per-cell calibration count n_c (80 ms, empirical-Bernstein, plug-in Γ_c). The 3-dataset subsample ($K=4$) and the 5-dataset K -sweep probe both follow the predicted $\propto 1/\sqrt{n_c}$ shrinkage; at large n_c the plug-in bound falls below the static realized violation rate (dashed) while staying valid (bound $\geq$ realized). The 5-dataset series is a scoped probe (single budget, ColBERT dropped for SciDocs), not a like-for-like extension of Table IV.

Three findings. (1) *The certificate is now non-vacuous*: the worst-case Hoeffding R^{cert} ranges 0.298–0.655 (min at $K=4$; empirical-Bernstein tightens $K=4$ to 0.263), down from ≈ 0.95 on the single collection. (2) *The policy strongly controls realized violation*: at 80 ms CWC incurs 0.001–0.002 vs. 0.112 for the strongest calibration-aware static selector ($\approx$ two orders of magnitude fewer) at a $\approx 4\%$ nDCG cost (0.609 vs. 0.636 at $K=4$). (3) *The gap closes with scale* (Figure 2): on the canonical workload the bound (≈ 0.30) still exceeds static’s realized 0.112 (a residual $\Gamma_c \hat{L}_c$ term, $\Gamma_c \approx 3$ –4.4), but a five-dataset probe (ArguAna+SciDocs, 3,654 queries, n_c tripled) drives the plug-in bound to $R^{\text{cert}}=0.041/0.061$ at $K=2/3$, below that probe’s static realized rate (0.063), following the $1/\sqrt{n_c}$ law: a worst-case bound dipping under an empirical rate, an operational milestone, not a certified-dominance theorem. The UCB- Γ variant stays vacuous even there; the bound exceeds the realized violation in every run.

What the bound certifies. R^{cert} is a distribution-free *worst-case ceiling* (violation rate $\leq R^{\text{cert}}$ w.p. $1 - \delta$ under bounded shift), the form an SLA needs (it holds even under adversarial-within-budget shift), not an estimate of the realized rate ($< 1\%$); its looseness is the price of that guarantee at finite

TABLE V

SENSITIVITY CERTIFICATE AT THE 80 MS BUDGET, $K=4$. R^{cert} AS A FUNCTION OF THE TRUE PER-CELL SHIFT BUDGET Γ (AS A MULTIPLE OF THE PLUG-IN $\hat{\Gamma} \approx 3.0$); Γ^* IS THE LARGEST TRUE BUDGET KEEPING $R^{\text{cert}} \leq \tau$. THE PLUG-IN BOUND EXCEEDS THE REALIZED VIOLATION AT EVERY ROW. REALIZED VIOLATION IS ≈ 0.001 THROUGHOUT.

$\Gamma/\hat{\Gamma}$	true Γ	R^{cert}	valid
0.5	1.50	0.155	✓
1.0	3.01	0.298	✓
1.5	4.51	0.393	✓
2.0	6.02	0.467	✓
2.5	7.52	0.541	✓
3.0	9.03	0.616	✓
4.0	12.0	0.764	✓
Tolerable budget: $\Gamma_{0.3}^* \approx 3.0$, $\Gamma_{0.5}^* \approx 6.7$, $\Gamma_{0.7}^* \approx 10.7$			

samples and shrinks with larger windows ($\approx 0.95 \rightarrow 0.30 \rightarrow 0.04$ above).

Sensitivity certificate: the tolerable shift budget. Because the plug-in Γ_c is the contract’s one unverified input (Section V), we turn it into an operational statement. Sweeping the true budget as a multiple of $\hat{\Gamma} \approx 3.0$ (R^{cert} is linear in Γ_c), Table V reports, for each SLA level τ , the largest true budget Γ^* keeping the certificate valid, a conditional SLA: *if the true shift stays below Γ^* , the window violation rate is at most τ* . For $\tau=0.5$ this tolerates $\Gamma^* \approx 6.7$ ($2.2\times$ the estimate), so the certificate degrades gracefully and the operator audits the margin rather than trusting a point estimate (realized violation stays ≈ 0.001).

Is the gain just the wrapper? A natural worry is that CWC’s edge comes from the abstain/certificate *wrapper* rather than from workload cells. We test this by running CWC with a single cell ($K=1$): it binds the one global constrained-utility-best plan and applies the same abstention and certificate; equivalently, the strongest static action wrapped in our machinery. At 80 ms this $K=1$ variant abstains on the entire window and retreats to the feasibility-best fallback (nDCG 0.586, $R^{\text{cert}}=0.157$), whereas full CWC ($K=4$) places safe high-quality plans per cell, needs no abstention, and reaches nDCG 0.604 at comparable realized violation (≈ 0.001). The workload cells, not the wrapper, are what preserve quality under the SLO.

D. Application: SLA admission control

The certificate is meant to be used, not just reported. We close the empirical section with a deployment scenario that turns the SLO-violation rate into the deciding criterion. An operator serving workload windows under a service-level agreement (SLA) admits a window’s deployed plan only if its realized eval-window SLO-violation rate meets the budget τ ; an admitted window earns its retrieval quality, a window that breaches the SLA earns none (it must fall back, and the quality gain is forfeited). The corresponding *admission utility* of a method on a block is therefore its mean nDCG@10 if the block’s realized violation is $\leq \tau$ and 0 otherwise. We evaluate this on *exactly the canonical 45-block study and the same*

TABLE VI

SLA ADMISSION CONTROL ON THE CANONICAL 45-BLOCK STUDY AT A $\tau=10\%$ SLA, OVER THE *same* SEVEN-METHOD POOL AS TABLE III. ADMISSION UTILITY IS MEAN nDCG@10 IF THE BLOCK’S REALIZED VIOLATION RATE IS $\leq \tau$, ELSE 0; LOWER RANK IS BETTER, “VIOL” IS THE MEAN REALIZED VIOLATION RATE. CWC RANKS FIRST AND HOLM-BEATS ALL SIX BASELINES (ONE-SIDED WILCOXON; FRIEDMAN $\chi^2=146.4$, $p=4.4 \times 10^{-29}$); NO VALUE IS HAND-SET.

Method	viol	nDCG@10	avg. rank	Holm p vs CWC
CWC (ours)	0.007	0.641	1.44	—
CostGreedy-cal	0.003	0.637	2.96	7.9×10^{-5}
Static-SPLADE	0.000	0.605	3.73	6.2×10^{-5}
Static-Qwen3	0.089	0.638	4.13	2.9×10^{-5}
StaticBest-cal	0.002	0.637	4.29	7.9×10^{-5}
Static-E5	0.000	0.542	4.67	1.9×10^{-8}
Static-ColBERTv2	0.583	0.584	6.78	1.5×10^{-8}

seven-method pool as Table III—the four SOTA operators (Qwen3-Embedding, SPLADE, ColBERTv2, E5) and the two calibration-aware selectors—so the comparison is apples-to-apples with the headline gate; only the per-block objective changes from constrained utility to SLA admission utility. We rank the seven methods per block and apply the same one-sided Holm-corrected Wilcoxon test as in Section VII-A. Table VI reports the resulting admission utilities, ranks and realized violation rates.

Under this deployment objective CWC is *average-rank first* (1.44 over 45 blocks) and Holm-beats all six baselines, including both calibration-aware selectors ($p \leq 8 \times 10^{-5}$), with a positive mean-utility margin against every one. The mechanism is the certificate’s design goal: CWC pairs the pool’s highest quality (nDCG@10 0.641) with a low realized violation (0.007), so its windows are almost always admitted, whereas Static-Qwen3 has comparable quality (0.638) but 0.089 violation and is rejected on tight-budget windows, and the calibration-aware selectors stay feasible only at lower quality. So the selectors that merely *tie* CWC on the unconstrained objective (Table III) *separate* from it once the SLA decides. The boundary is honest: at a tighter $\tau=5\%$ SLA CWC stays rank-first (1.71) but its edge narrows to borderline ($p \approx 0.048$) and is no longer a mean-utility win (CWC is itself occasionally rejected at 5%); the advantage is clearest at moderate SLAs.

E. Systems overhead on a real hybrid stack

To confirm CWC is a lightweight optimizer layer, we integrate it with a real hybrid-search stack—a BM25 index, a dense HNSW index (HNSWlib over E5-base), and a cross-encoder reranker (MiniLM)—over NFCorpus (3,633 docs, 323 queries) on the RTX 4090 of Section VI. Per-query plan latencies span 0.16 ms (dense HNSW) to 13.6 ms (dense+rerank@20); every latency in this subsection is a wall-clock mean over the 323 NFCorpus test queries, measured with `time.perf_counter` on the single RTX 4090 of Section VI after a five-query warm-up, with all indexes resident. Against this, CWC adds a one-time *compile* of 47.6 ms per window (≈ 0.15 ms amortized/query) and 0.17 ms/query

routing (cell assignment + dispatch); telemetry probes cost 2.5 ms/query and reuse the retrieval scores. The optimizer layer is thus sub-millisecond per query and negligible against retrieval and reranking, so the certified gains come at essentially no serving cost.

F. NFCorpus pilot (single-collection, preliminary)

This pilot was run *first* chronologically and motivated the design; we report it *after* the canonical multi-dataset result because it uses a single collection and a local action catalog, and so is the weaker evidence of the two. Readers who prefer development order may read this subsection before Section VII-A. As an initial single-collection study, WCR passes the 30-block Rank-1 gate (ranks over a pool of 26 methods; Figure 3a). The candidate reaches average rank 3.72, ahead of QueryOnly-RF (5.82), CostGreedy-cal and StaticBest-cal (6.12), and ScalarTelemetry-RF (7.87); Holm-adjusted p -values are 0.013 against QueryOnly-RF, 0.012 against the two calibration-aware selectors, and $\leq 1.6 \times 10^{-3}$ against every static operator. The rank result reflects higher constrained utility, not only a favorable rank statistic (Figure 3b): WCR has mean constrained score 0.3537, satisfaction 0.9986, latency 14.27 ms, and regret 0.0488, dominating QueryOnly-RF on score and satisfaction and ScalarTelemetry-RF (lower latency but lower score, higher regret) on the objective.

G. Pilot ablations, controls, and stability

Cell count (Figure 3c). The result is not tied to $k=6$: in the k -sweep $k=5$ also passes (rank 3.78, score 0.3536, satisfaction 0.9972, regret 0.0489), and the unsupervised elbow selector picks $k=5$ in all 30 blocks and passes (Holm $p=0.014$ vs. QueryOnly-RF), reducing cherry-pick risk; the silhouette selector fails (rank 4.35, three failed comparisons), marking the boundary that unsupervised k selection helps only when the rule matches the constrained-utility geometry.

Feature families and controls (Figure 3d). Full telemetry passes with zero failures; removing dense features weakens but does not destroy the signal (rank 5.52, four failures); metadata-only, no-BM25, no-probe, no-overlap, and query-only all fail (query-only rank 6.38, nine failures; metadata-only collapses to $k=1$). Train-only (4.75), random (7.32), and query-only (6.38) $k=6$ controls all fail, confirming the benefit comes from unlabeled workload structure plus retrieval telemetry, not random partitioning or query-only routing. *Stability report* (the diagnostic of Section IV-D, distinct from the certificate of Section V): across 180 cells and 1290 decisions the mean bootstrap support is 0.813, but the weak slice (selectivity 0.5, 80 ms) drops to 0.610 with risky-cell share 0.533, and flagged queries have higher regret (0.0695 vs. 0.0329), so it isolates the fragile part of the frontier. *External operators*. Static use of the strong imported operators is not competitive under the constrained objective (SPLADE-pre rank 12.0, Qwen3-pre 17.1, ColBERTv2-pre 21.0): a strong retriever used statically is not the same as optimizing the physical plan under filter

and latency constraints, so the pilot claim is optimizer-level, not “WCR is the strongest retriever”.

H. Cross-Dataset Robustness Audit

A secondary local-catalog robustness audit over SciFact, NFCorpus, and FiQA (360 selectivity-budget-seed blocks, pool of 16 methods) corroborates the main result: WCR obtains the best average rank (4.16), Holm-significantly beating QueryOnly-RF (5.05, $p=5.1 \times 10^{-8}$), CostGreedy-cal (5.61), ScalarTelemetry-RF (6.20), and every fixed static operator ($p \leq 3.9 \times 10^{-14}$). As in the canonical study, the comparison against StaticBest-cal (4.27) is *not* significant ($p=0.185$): the audit supports robustness beyond NFCorpus but not universal dominance over every calibration-aware selector.

VIII. DISCUSSION AND CONCLUSION

CWC takes a workload-level view of hybrid search: not a new retriever, but a compiler that selects among existing physical plans more effectively than static policies, cost-greedy selection, or scalar/query-only routing, and—uniquely among our baselines—backs each deployment decision with a distribution-free certificate on the SLO-violation rate. This is what the paper solves: it turns plan selection from “deploy and hope it generalizes” into an auditable, admission-ready decision over unchanged operators.

We state the limits plainly. Even in the canonical study CWC only *ties* the strongest calibration-aware selectors on mean utility (its differentiator is the certificate, not raw rank), separating from them only once the SLA is the deciding criterion (Section VII-D); the audit also only ties StaticBest-cal; and NV-Embed/RankZephyr are not executed as true actions. Under the plug-in contract the bound *empirically upper-bounds the realized violation in every evaluated run* but is *loose* on a 100-query collection (the $\Gamma_c/\sqrt{n_c}$ term keeps the bound near one), and Γ_c is an estimated plug-in rather than a theorem-valid quantity; its tightness, not its validity, improves with larger windows. These boundaries match the intended deployment setting: CWC targets workload-window admission and SLO-aware plan selection under a fixed action catalog, not the replacement of end-to-end retriever engineering or the certification of arbitrary shift magnitudes.

Future work tightens the end-to-end Γ_c UCB (Remark 1) and extends CWC to more operators and streaming workloads.

DATA AVAILABILITY STATEMENT

The datasets are public BEIR benchmarks [25] (SciFact, FiQA-2018, NFCorpus), available via the BEIR repository (<https://github.com/beir-cellar/beir>) and the Hugging Face Hub (<https://huggingface.co/BeIR>). The complete artifact is public at <https://github.com/asdasaffq/cwc>: the full operator-execution and action-row generation pipeline, every script behind every number in this paper, the compressed action rows needed to reproduce all tables without a GPU, and the end-to-end serving harness.

Workload-compiled retrieval is rank-first under constrained hybrid search

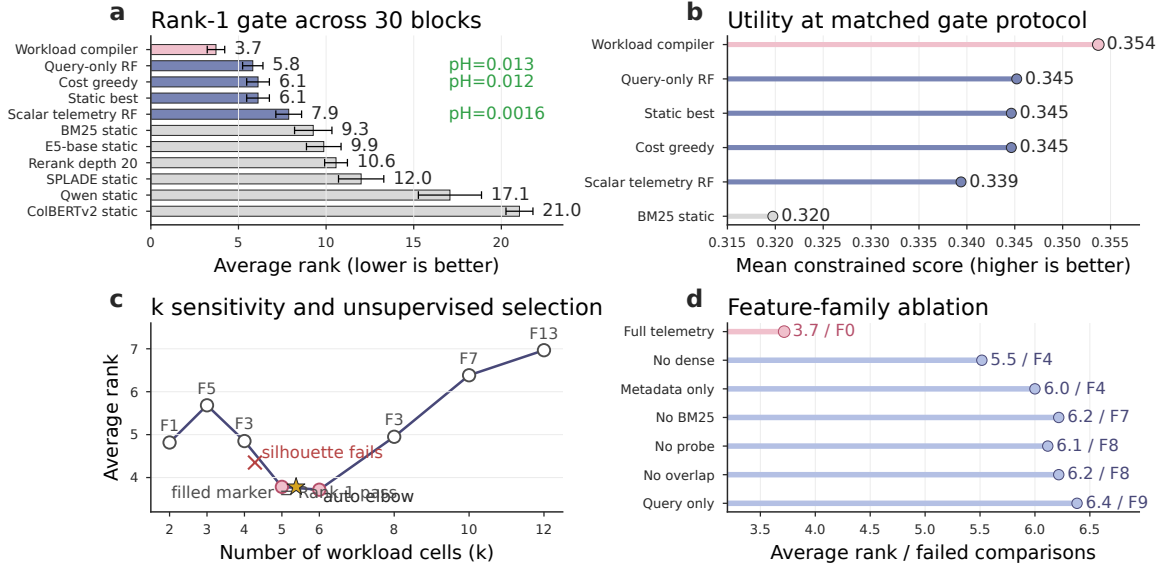


Fig. 3. *Pilot illustration* (single-collection NFCorpus, 30 blocks; preliminary, not the headline). (a) Average rank (lower better; error bars are the SEM of block-level ranks; Holm-adjusted tests vs. baselines). (b) Mean constrained score under the same gate. (c) Cell-count sensitivity: filled markers pass the Rank-1 gate; the elbow selector picks $k=5$ and passes, silhouette is a negative control. (d) Feature-family ablation at $k=6$: removing retrieval-telemetry families causes gate failures, while full telemetry stays first-ranked.

CONFLICT OF INTEREST

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

AI-GENERATED CONTENT ACKNOWLEDGMENT

The authors used ChatGPT for language editing. All technical claims, experimental results, citations, and final text were verified by the authors, who take full responsibility for the content.

REFERENCES

- [1] S. Robertson and H. Zaragoza, *The probabilistic relevance framework: BM25 and beyond*. Now Publishers Inc, 2009, vol. 4.
- [2] L. Wang, N. Yang, X. Huang, B. Jiao, L. Yang, D. Jiang, R. Majumder, and F. Wei, “Text embeddings by weakly-supervised contrastive pre-training,” *arXiv preprint arXiv:2212.03533*, 2022.
- [3] T. Formal, B. Piwowarski, and S. Clinchant, “Splade: Sparse lexical and expansion model for first stage ranking,” in *Proceedings of the 44th International ACM SIGIR Conference on Research and Development in Information Retrieval*, 2021, pp. 2288–2292.
- [4] K. Santhanam, O. Khattab, J. Saad-Falcon, C. Potts, and M. Zaharia, “Colbertv2: Effective and efficient retrieval via lightweight late interaction,” in *Proceedings of the 2022 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, 2022, pp. 3715–3734.
- [5] G. V. Cormack, C. L. Clarke, and S. Buettcher, “Reciprocal rank fusion outperforms condorcet and individual rank learning methods,” in *Proceedings of the 32nd international ACM SIGIR conference on Research and development in information retrieval*, 2009, pp. 758–759.
- [6] R. Pradeep, S. Sharifmoghaddam, and J. Lin, “Rankzephyr: Effective and robust zero-shot listwise reranking is a breeze!” *arXiv preprint arXiv:2312.02724*, 2023.
- [7] P. G. Selinger, M. M. Astrahan, D. D. Chamberlin, R. A. Lorie, and T. G. Price, “Access path selection in a relational database management system,” in *Proceedings of the 1979 ACM SIGMOD international conference on Management of data*, 1979, pp. 23–34.
- [8] G. Graefe and W. J. McKenna, “The volcano optimizer generator: Extensibility and efficient search,” in *Proceedings of IEEE 9th international conference on data engineering*. IEEE, 1993, pp. 209–218.
- [9] S. Chaudhuri and V. Narasayya, “Autoadmin “what-if” index analysis utility,” *ACM SIGMOD Record*, vol. 27, no. 2, pp. 367–378, 1998.
- [10] R. Avnur and J. M. Hellerstein, “Eddies: Continuously adaptive query processing,” in *Proceedings of the 2000 ACM SIGMOD international conference on Management of data*, 2000, pp. 261–272.
- [11] Z. Yi, Y. Tian, Z. G. Ives, and R. Marcus, “Low rank learning for offline query optimization,” *Proceedings of the ACM on Management of Data*, vol. 3, no. 3, pp. 1–26, 2025.
- [12] J. Tao, N. Maus, H. Jones, Y. Zeng, J. R. Gardner, and R. Marcus, “Learned offline query planning via bayesian optimization,” *Proceedings of the ACM on Management of Data*, vol. 3, no. 3, pp. 1–29, 2025.
- [13] J. T.-H. Hsu, “Searchgym: A modular infrastructure for cross-platform benchmarking and hybrid search orchestration,” *arXiv preprint arXiv:2603.04402*, 2026.
- [14] Y. Xu, M. Ying, W. Guo, and Z. Wei, “Two-stage risk control with application to ranked retrieval,” *arXiv preprint arXiv:2404.17769*, 2024.
- [15] Y. Xu, W. Guo, and Z. Wei, “Selective conformal risk control,” *arXiv preprint arXiv:2512.12844*, 2025.
- [16] A.-G. Chifu, S. Déjean, M. Garouani, J. Mothe, D. Ortiz, and M. Z. Ullah, “Uncovering the limitations of query performance prediction: Failures, insights, and implications for selective query processing,” *ACM Transactions on Information Systems*, 2025.
- [17] J. Mothe and M. Z. Ullah, “Selective query processing: A risk-sensitive selection of search configurations,” *ACM Transactions on Information Systems*, vol. 42, no. 1, pp. 1–35, 2023.
- [18] H. Liu, S. Giridhara, and I. Sabek, “Conformal prediction for verifiable learned query optimization,” *arXiv preprint arXiv:2505.02284*, 2025.
- [19] R. Marcus, P. Negi, H. Mao, N. Tatbul, M. Alizadeh, and T. Kraska, “Bao: Making learned query optimization practical,” in *Proceedings of the 2021 International Conference on Management of Data*, 2021, pp. 1275–1288.
- [20] L. Li, W. Chu, J. Langford, and R. E. Schapire, “A contextual-bandit approach to personalized news article recommendation,” in *Proceedings of the 19th international conference on World wide web*, 2010, pp. 661–670.
- [21] J. Chen, S. Xiao, P. Zhang, K. Luo, D. Lian, and Z. Liu, “M3-embedding: Multi-linguality, multi-functionality, multi-granularity

text embeddings through self-knowledge distillation,” *arXiv preprint arXiv:2402.03216*, 2024.

- [22] Y. Zhang, M. Li, D. Long, X. Zhang, H. Lin, B. Yang, P. Xie, A. Yang, D. Liu, J. Lin *et al.*, “Qwen3 embedding: Advancing text embedding and reranking through foundation models,” *arXiv preprint arXiv:2506.05176*, 2025.
- [23] C. Lee, R. Roy, M. Xu, J. Raiman, M. Shoeybi, B. Catanzaro, and W. Ping, “Nv-embed: Improved techniques for training llms as generalist embedding models,” in *International Conference on Learning Representations*, vol. 2025, 2025, pp. 79 310–79 333.
- [24] O. Khattab and M. Zaharia, “Colbert: Efficient and effective passage search via contextualized late interaction over bert,” in *Proceedings of the 43rd International ACM SIGIR conference on research and development in Information Retrieval*, 2020, pp. 39–48.
- [25] N. Thakur, N. Reimers, A. Rücklé, A. Srivastava, and I. Gurevych, “Beir: A heterogenous benchmark for zero-shot evaluation of information retrieval models,” *arXiv preprint arXiv:2104.08663*, 2021.